

EKF Parameters

Theory $\leftrightarrow$ Practice

Each Knob Explained:
Intuition, Math, Effect

Husarion Lynx — robot_localization dual-EKF + NavSat

Multi-Sensor Fusion Project

May 9, 2026

Abstract

Companion guide to the EKF Deep Dive document. Where the deep dive built theory bottom-up, this document goes parameter-by-parameter through the actual `robot_localization` configuration used in the Husarion Lynx project (`dual_ekf_navsat.yaml`), explaining for every knob: (i) what the parameter does mechanically inside the EKF; (ii) the mathematical equation it appears in; (iii) the intuition — what increasing or decreasing it physically means; (iv) the effect on filter behavior; (v) typical failure modes when misconfigured; (vi) how to tune it. Notation throughout follows Thrun-Burgard-Fox: state mean μ_t , covariance Σ_t , motion noise covariance R_t , measurement noise covariance Q_t . Note: the `robot_localization` YAML uses the engineering convention where the YAML key `process_noise_covariance` stores what we call R_t in Thrun notation.

Contents

1 The Three EKFs in this Project

The Husarion Lynx setup runs three EKF instances plus one transform node. Understanding which one each parameter belongs to is the first step.

Node	Frame	Inputs	Outputs
ekf_filter_node_odom (EKF #1, Local)	world_frame: odom	wheel + IMU + VO + LiDAR + control	/odometry/filtered, TF odom→base_link
ekf_filter_node_map (EKF #2, Global)	world_frame: map	wheel + IMU + VO + LiDAR (NO GPS)	/odometry/filtered_map, TF map→odom
navsat_transform	—	GPS (NavSatFix) + IMU + /odometry/filtered	/odometry/gps (analysis only)

Critical Architectural Decision

This project deliberately departs from the canonical `robot_localization` dual-EKF + NavSat tutorial. **GPS is NOT fused into EKF #2.** Both EKFs receive identical sensor inputs (wheel + IMU + VO + LiDAR), differing only in `world_frame`.

Reason: GPS noise injected through cross-covariance terms $\Sigma_{x,\psi}$ and $\Sigma_{y,\psi}$ caused sustained map frame yaw drift of $\sim 3.56/s$ in earlier configurations. Without a strong absolute yaw observation (no magnetometer in this Gazebo IMU, no dual-antenna RTK), nothing in the filter can fight the leak. The covariance cross-term problem is the single most important reason to read this document carefully before re-enabling GPS in EKF #2.

GPS still flows through the pipeline (`navsat_transform_node` runs, `/odometry/gps` is published) but neither EKF subscribes to that topic. GPS is available for offline analysis notebooks comparing filter output to GPS truth, but does *not* influence the live state estimate.

See Section ?? for the full mathematical derivation, numerical example, trade-offs, and alternatives.

Why this dual-EKF then? With both EKFs receiving identical inputs, the resulting estimates converge to nearly the same value, so $T_{\text{map}}^{\text{odom}} \approx I$. EKF #1 publishes `odom→base_link` for the controller (smooth, continuous). EKF #2 publishes `map→odom` (near-identity, also smooth in this setup). The map frame is *anchored at the robot's spawn position* via sensor consensus, not GPS.

Sensor noise measured for this hardware (from `plots/summary.md`):

- GPS: $\sigma_{\text{lat}}^2 = 0.263 \text{ m}^2$, $\sigma_{\text{lon}}^2 = 0.245 \text{ m}^2$, $\sigma_{\text{alt}}^2 = 1.230 \text{ m}^2$
- IMU gyro: $\sigma_x^2 = 2.26 \times 10^{-4}$, $\sigma_y^2 = 1.46 \times 10^{-3}$, $\sigma_z^2 = 4.16 \times 10^{-2} \text{ (rad/s)}^2$
- IMU accel: $\sigma_x^2 = 0.97$, $\sigma_y^2 = 0.57$, $\sigma_z^2 = 0.69 \text{ (m/s}^2)^2$
- Wheel odom: `pose_cov = 0.001`, `twist_cov = 0.001` (from `diff_drive_controller`)
- Map frame stability (current config, GPS excluded): $\pm 0.1 \text{ m}$, ± 1 over 2 minutes

Notation reminder. In the YAML keys, `process_noise_covariance` corresponds to Thrun's R_t (motion uncertainty). The covariance fields inside each `nav_msgs/Odometry` or `sensor_msgs/Imu` message correspond to Thrun's Q_t (measurement uncertainty).

2 State Vector Refresher

Every `ekf_node` in `robot_localization` maintains a 15-dimensional state:

$$\mu_t = \left[\underbrace{x, y, z}_{\text{position}}, \underbrace{\phi, \theta, \psi}_{\text{roll, pitch, yaw}}, \underbrace{\dot{x}, \dot{y}, \dot{z}}_{\text{linear vel}}, \underbrace{\dot{\phi}, \dot{\theta}, \dot{\psi}}_{\text{angular vel}}, \underbrace{\ddot{x}, \ddot{y}, \ddot{z}}_{\text{linear accel}} \right]^\top$$

Every parameter that takes a 15-element list (e.g., `process_noise_covariance` diagonal, `initial_estimate_cov` diagonal) maps element-wise to this ordering. Memorise the order: position, RPY, linear velocities, angular velocities, linear accelerations.

Index reference:

Idx	Component	Idx	Component	Idx	Component
0	x	5	ψ (yaw)	10	$\dot{\theta}$
1	y	6	$\dot{x}$ (vx)	11	$\dot{\psi}$ (vyaw)
2	z	7	$\dot{y}$ (vy)	12	$\ddot{x}$ (ax)
3	ϕ (roll)	8	$\dot{z}$ (vz)	13	$\ddot{y}$ (ay)
4	θ (pitch)	9	$\dot{\phi}$ (vroll)	14	$\ddot{z}$ (az)

3 Frame Parameters

These define the coordinate frames in the TF tree.

map_frame

Type: string **Project value:** map

What: Name of the global, world-fixed reference frame.

Math role: The frame in which μ_t is expressed when `world_frame: map`. All transforms involving GPS, ENU coordinates terminate here.

Intuition: “The unmoving big-picture frame.” If you stood outside the robot with a god-view, this is your frame. Origin is set by GPS datum or pre-defined.

Effect: Only meaningful for the *global* EKF (EKF #2). Local EKF still has it set but ignores it (since `world_frame: odom`). Must match across all nodes that share a TF tree.

Failure mode: Misnaming silently disconnects the TF tree — planner finds no path to `base_link`.

odom_frame

Type: string **Project value:** odom

What: Name of the locally drifting world-fixed reference frame.

Math role: The frame for μ_t when `world_frame: odom`. The local EKF (EKF #1, EKF #3) operates here.

Intuition: “Where the robot thinks it is, ignoring GPS.” Drifts with time but is locally smooth and continuous — exactly what the controller needs.

Effect: The local EKF publishes `odom→base_link`. The global EKF publishes `map→odom`. The composition gives global pose.

Failure mode: If two nodes claim to publish `odom→base_link`, TF tree breaks (tf complains about multiple parents).

base_link_frame

Type: string **Project value:** base_link

What: Frame rigidly attached to robot body.

Math role: All sensor measurements are transformed into this frame using static transforms from URDF before fusion. The motion model $g(u_t, \mu_{t-1})$ predicts the pose of `base_link` relative to the world frame.

Intuition: “The robot itself.” Convention: $+x$ forward, $+y$ left, $+z$ up (right-hand rule).

Effect: All input sensor messages must have their `frame_id` resolvable to `base_link` via static TF. Wrong sensor frame $\rightarrow$ measurement applied at wrong rotation/translation $\rightarrow$ filter biases.

world_frame

Type: string **Project value:** `odom` (EKF #1, #3) / `map` (EKF #2)

What: The frame in which the EKF maintains its state estimate.

Math role: Determines which TF transform the node publishes:

- `world_frame == odom_frame`: publishes `odom` $\rightarrow$ `base_link`
- `world_frame == map_frame`: publishes `map` $\rightarrow$ `odom` (computed as $T_{\text{map}}^{\text{base}} \cdot (T_{\text{odom}}^{\text{base}})^{-1}$)

Intuition: *This is the most important frame parameter.* Sets which role the EKF plays — local smoother or global corrector.

Effect: The dual-EKF architecture exists exactly because of this parameter. Two filters, two values, decoupled transforms.

Failure mode: Both EKFs running with same `world_frame` $\rightarrow$ TF conflict.

4 Timing Parameters

frequency

Type: double, Hz **Project value:** 50.0 (EKFs) / 10.0 (navsat)

What: How often the EKF runs its predict step and publishes output.

Math role: Determines Δt used in:

- Motion model integration: $\bar{\mu}_t = g(u_t, \mu_{t-1})$ uses $\Delta t = 1/\text{frequency}$
- Process noise scaling: $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + R_t \cdot \Delta t$
- Jacobian evaluation point and step size

Intuition: “How often does the filter take a heartbeat?” Higher = smoother output, more CPU. Lower = jerkier, lighter.

Effect of increasing (e.g., 50 $\rightarrow$ 100 Hz):

- Smaller Δt per step $\rightarrow$ less linearisation error per step (Taylor expansion more accurate over smaller interval)
- More frequent predictions absorb sensor jitter better
- CPU load increases linearly
- Diminishing returns past sensor rates — predicting faster than sensors update gains nothing

Effect of decreasing (e.g., 50 $\rightarrow$ 10 Hz):

- Larger Δt per step $\rightarrow$ more linearisation error
- Output less smooth — controller may oscillate
- For nonlinear motion ($\omega \neq 0$), arc segments become noticeable

Project rationale: 50 Hz matches the IMU rate (typically 100 Hz raw $\rightarrow$ filtered to 50). Higher frequency than fastest sensor wastes CPU. `navsat_transform` runs at 10 Hz to match the GPS update rate.

Tuning rule: Set to the rate of your fastest critical sensor, or what the controller expects (Nav2 typically wants ≥ 30 Hz odometry).

sensor_timeout**Type:** double, seconds **Project value:** 0.1**What:** If no measurement arrives within this window, the EKF skips correction this cycle and just predicts.**Math role:** Controls when the filter falls into *prediction-only* mode. After timeout:

$$\mu_t = \bar{\mu}_t = g(u_t, \mu_{t-1}), \quad \Sigma_t = \bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + R_t$$

without any correction step. Uncertainty grows unboundedly during this period.

Intuition: “How long do I wait for a sensor before giving up?” Trade-off: too short = drops occasional late messages; too long = filter waits for stale data, output stale.**Effect:**

- Short (0.05 s): aggressive, predicts often, robust to dropped messages
- Long (0.5 s): conservative, may freeze on first sensor disconnect

Project value 0.1 s: matches $1/\text{frequency} = 1/50 = 0.02$ s with 5x slack. Sensor offline for > 0.1 s $\rightarrow$ predict-only.**Failure mode:** Set too low with jittery sensors $\rightarrow$ filter stutters.**transform_time_offset****Type:** double, seconds **Project value:** 0.0**What:** Adds a future timestamp offset to published TF, so other nodes can do future-time lookups.**Math role:** Pure metadata — shifts the timestamp on the published **TransformStamped**. State estimate unchanged.**Intuition:** “Backdate or future-date my TF transforms.” Useful when consumers want to predict slightly into the future.**Effect:**

- 0.0 (project): TF timestamp = filter timestamp. Standard.
- 0.1: published TF claims to be valid for 0.1 s into future. Reduces “extrapolation” warnings in TF lookups by Nav2.

transform_timeout**Type:** double, seconds **Project value:** 0.0**What:** How long the TF buffer holds onto the previous transform if a new one is delayed.**Math role:** TF interpolation/extrapolation buffer.**Effect:** 0.0 means no timeout — always serve latest. Tweak if downstream consumers are missing transforms.

5 Mode Parameters

two_d_mode**Type:** bool **Project value:** true**What:** Constrains state to planar motion. $z, \phi, \theta, \dot{z}, \dot{\phi}, \dot{\theta}, \ddot{z}$ are clamped to zero.**Math role:** Effectively reduces the 15-D state to an 8-D state $[x, y, \psi, \dot{x}, \dot{y}, \dot{\psi}, \ddot{x}, \ddot{y}]^\top$. Internally implemented by zero-ing rows/columns 2, 3, 4, 8, 9, 10, 14 of $\bar{\Sigma}_t$ after each update.

Intuition: “This robot doesn’t fly or fall.” Skips degrees of freedom your robot physically can’t have.

Effect of true:

- Prevents IMU pitch/roll noise from corrupting yaw estimate via cross-correlation
- Prevents drift in z from accumulated accelerometer error
- Reduces effective state dimension $\rightarrow$ better-conditioned filter

Effect of false:

- Full 6-DOF tracking — needed for drones, climbing robots, ramps
- Roll/pitch from IMU integrated. Useful but introduces noise into ψ via Coriolis/-correlation

Project rationale: Lynx is a ground robot on flat-ish terrain. **true** is correct. If running on steep slopes or stairs, set to **false**.

Failure mode: Setting **true** when robot actually pitches significantly $\rightarrow$ accelerometer measurements get rotated incorrectly (gravity not removed properly), $\rightarrow$ horizontal acceleration biases.

publish_tf

Type: bool **Project value:** true

What: Whether the EKF publishes the world_frame $\rightarrow$ base_link_frame TF.

Effect: Both EKFs in this project publish their TF (one publishes odom $\rightarrow$ base_link, the other map $\rightarrow$ odom). Setting **false** means another node must publish that TF (rare).

Failure mode: Two nodes both publishing same TF $\rightarrow$ TF tree warnings, intermittent values.

publish_acceleration

Type: bool **Project value:** false

What: Publishes filtered linear acceleration on /accel/filtered.

Effect: If **true**, additional message published with $\ddot{x}, \ddot{y}, \ddot{z}$ from state. If consumer cares (e.g., for vibration analysis), enable. Otherwise **false** saves bandwidth.

print_diagnostics

Type: bool **Project value:** true

What: Publishes diagnostic info on /diagnostics including measurement queue lengths, transform errors, etc.

Effect: Enabling helps tuning. No runtime cost worth worrying about.

debug

Type: bool **Project value:** false

What: Verbose logging of every measurement, prediction, and correction.

Effect: **true** produces large debug files. Useful for tuning or post-mortem. Disable in production.

6 Sensor Input Configuration

For each sensor (odom0, odom1, imu0, ...), several parameters control how its data enters the filter.

6.1 Topic Name (odomN, imuN, poseN, twistN)

odom0 / imu0 / ...

Type: string **Project values:** /diff_drive_controller/odom, /imu/data, /odometry/gps, /odometry/visual, /odometry/lidar

What: ROS topic the EKF subscribes to.

Math role: Each topic provides one stream of measurements $z_t^{(s)}$ with covariance $Q_t^{(s)}$ (where s indexes the sensor). The filter applies the standard EKF correction once per arriving message:

$$K_t = \bar{\Sigma}_t H_t^\top (H_t \bar{\Sigma}_t H_t^\top + Q_t^{(s)})^{-1}$$

Effect: Each sensor enters as an independent correction step. Order of subscriber declaration in YAML doesn't matter; messages are processed in timestamp order.

Best practice: One sensor per topic. Don't merge two encoders into one topic — their covariances become unrecoverable.

6.2 15-Bit Configuration Array

This is the most important per-sensor parameter. It selects which state components the sensor measures.

odomN_config / imuN_config

Type: list of 15 booleans **Project examples below**

What: 15-element boolean array selecting which state dimensions this sensor contributes to. Mapping matches the state vector indexing.

$$\text{config} = [b_x, b_y, b_z, b_\phi, b_\theta, b_\psi, b_{\dot{x}}, b_{\dot{y}}, b_{\dot{z}}, b_{\dot{\phi}}, b_{\dot{\theta}}, b_{\dot{\psi}}, b_{\ddot{x}}, b_{\ddot{y}}, b_{\ddot{z}}]$$

Math role: Defines the observation matrix H_t (or C_t for linear case). Each **true** bit becomes a row of H_t that picks out the corresponding state element. Example: if config selects only $\dot{x}$ and $\dot{\psi}$:

$$H_t = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$

The measurement equation becomes:

$$z_t = \begin{bmatrix} \dot{x} \\ \dot{\psi} \end{bmatrix} + \delta_t$$

The corresponding entries of the message's covariance matrix are extracted to form Q_t for this update.

Intuition: “Tell the filter what each sensor knows.” A wheel encoder doesn't measure absolute position; only velocities. A GPS doesn't measure orientation. An IMU doesn't measure position.

Project values walked through:

Wheel odometry (EKF #1, #2, #3):

```
1 odom0_config: [false, false, false,      # x, y, z           NO
2                false, false, false,      # roll, pitch, yaw NO
3                true,  false, false,       # vx, vy, vz        vx YES
4                false, false, true,        # vroll, vp, vyaw   vyaw YES
5                false, false, false]       # ax, ay, az        NO
```

Why: Wheel encoders give body-frame linear velocity (forward only for diff-drive: $\dot{y}_{\text{body}} = 0$) and yaw rate. Position from `diff_drive_controller` drifts; we let the EKF integrate velocities itself, fusing with IMU.

IMU (all EKFs):

```
1 imu0_config: [false, false, false,      # NO position
2               true,  true,  true,       # roll, pitch, yaw YES
3               false, false, false,     # NO linear vel
4               true,  true,  true,       # ALL angular vels YES
5               true,  true,  true]      # ALL linear accels YES
```

Why: IMU provides absolute roll/pitch from gravity (always reliable), yaw from gyro integration (drifts but high-rate), all angular velocities (raw gyro), and linear accelerations (with gravity removed). Note: yaw can be problematic if magnetometer is bad — some setups disable bit #5.

GPS via navsat (EKF #2):

```
1 odom1_config: [true,  true,  false,     # x, y YES (planar)
2                false, false, false,
3                false, false, false,
4                false, false, false,
5                false, false, false]
```

Why: GPS gives only absolute (x, y) in map frame. No orientation, no velocity. z disabled (`zero_altitude` on navsat).

Visual / LiDAR odometry (EKF #3):

```
1 odom1_config: [true,  true,  false,     # x, y YES
2                false, false, true,      # yaw YES
3                false, false, false,
4                false, false, false,
5                false, false, false]
```

Why: Both VO and LO produce 6-DOF poses. In 2D mode we use (x, y, ψ) . Velocities not enabled to avoid double-counting (the wheel odom already provides those).

Failure mode: Enabling both pose and twist of the same sensor → filter receives a measurement and its time-derivative as “independent,” artificially over-confident.

6.3 Differential Mode

`odomN_differential`

Type: bool **Project value:** false

What: If `true`, the filter treats the difference between consecutive messages as a relative pose change rather than an absolute pose.

Math role: Instead of using $z_t = h(\bar{\mu}_t)$ as absolute pose, it uses:

$$z_t \rightarrow (z_t - z_{t-1}) + \mu_{t-1}^{(\text{pose})}$$

i.e., the new pose is the previous filter pose plus the measurement increment.

Intuition: “Use this sensor’s deltas, not its absolute readings.” Useful when:

- Two sources both report absolute pose but disagree on origin (e.g., two SLAM systems)
- You want to avoid GPS-induced jumps in the local filter

Effect:

- `false` (project): use absolute readings. Standard for one trusted source per modal-

ity.

- **true**: differentiate first. Removes constant offsets. Doubles noise (subtraction increases variance).

Project rationale: **false** everywhere. Wheel odom feeds velocities (already differential by nature). GPS feeds map-frame position which is what we want. Visual/LiDAR feed pose deltas via separate node integration.

Failure mode: Setting **true** on velocity feeds re-differentiates them → acceleration measurements with huge variance.

odomN_relative

Type: bool **Project value:** **false**

What: If **true**, treats the first message as the origin — subsequent messages are offsets from that first pose.

Effect: Useful when sensor reports in its own arbitrary frame. Project does not need this since wheel odom starts at origin and GPS starts at datum.

6.4 Queue and Latency

odomN_queue_size

Type: int **Project value:** 10

What: ROS subscriber queue size for buffering incoming messages.

Effect: Larger absorbs bursts. If sensor publishes at 50 Hz and EKF processes at 50 Hz, queue of 10 holds 0.2 s buffer.

Failure mode: Queue too small + slow processing → messages dropped (visible in /diagnostics).

odomN_nodelay

Type: bool **Project value:** **true/false** (varies)

What: Sets TCP_NODELAY on the subscription, disabling Nagle's algorithm to send small messages immediately.

Effect: **true** reduces latency for small high-rate streams (wheel odom). **false** default for IMU which already batches.

7 Measurement Rejection Thresholds

These implement Mahalanobis gating — rejecting measurements that are too far from the predicted mean.

imuN / odomN _pose_rejection_threshold (and _twist / _linear_acceleration variants)

Type: double, Mahalanobis distance squared **Project values:** 0.8 (IMU), 3.0 (GPS pose)

What: Maximum Mahalanobis distance for accepting a measurement. Larger distance → measurement rejected as outlier.

Math role: For each candidate measurement z_t , compute:

$$d_M^2 = (z_t - h(\bar{\mu}_t))^T S_t^{-1} (z_t - h(\bar{\mu}_t)), \quad S_t = H_t \bar{\Sigma}_t H_t^T + Q_t$$

Reject if $d_M^2 > \text{threshold}$.

Intuition: “How surprised do I have to be before I assume the sensor is lying?” Smaller = stricter, more rejections. Larger = more tolerant, accepts noisy data.

Statistical interpretation: Under the assumption that innovation is zero-mean Gaussian with covariance S_t , d_M^2 follows χ_k^2 where k is measurement dimension. Common thresholds:

k	95% threshold	99% threshold
1	3.84	6.63
2	5.99	9.21
3	7.81	11.34
6	12.59	16.81

Project values explained:

- IMU 0.8: very strict. Reflects high confidence in IMU — big residual = something else wrong.
- GPS 3.0: moderate. GPS is noisy by nature; allow some surprise before rejecting.

Effect of decreasing (e.g., 3.0 $\rightarrow$ 1.0):

- More GPS measurements rejected as outliers
- Faster recovery from rare GPS spikes (multipath, etc.)
- BUT: filter under-corrects when state has genuinely diverged — can lock to drifted estimate

Effect of increasing (e.g., 3.0 $\rightarrow$ 100.0):

- All measurements accepted
- Outliers contaminate state
- Useful initially during tuning; tighten later

Failure mode: Setting too low + initial covariance set very small (overconfident initial state) $\rightarrow$ all measurements rejected, filter drifts forever.

8 Process Noise Covariance R_t

This is the most-tuned parameter. It controls how much the filter “trusts itself.”

`process_noise_covariance`

Type: 15×15 matrix (225 floats), specified row-major **Project values:** see below

What: Covariance matrix R_t added to predicted covariance each step. Represents unmodelled state changes per unit time.

Math role: In the predict step:

$$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t \cdot \Delta t$$

(robot_localization scales R by Δt internally so values are per-second.)

Intuition: “How much can the world change between predictions, beyond what my motion model captures?” Models:

- Slip and unmodelled dynamics
- Wheel radius variation
- Calibration error

- External forces (wind, ground irregularities)

Effect of increasing (e.g., $R_{xx} : 0.05 \rightarrow 0.5$):

- Predicted covariance $\bar{\Sigma}_t$ grows faster
- Kalman gain $K_t = \bar{\Sigma}_t H_t^\top S_t^{-1}$ grows
- Filter trusts measurements more, follows them faster
- Output noisier (less smoothing)

Effect of decreasing (e.g., $R_{xx} : 0.05 \rightarrow 0.005$):

- Predicted covariance grows slowly
- Kalman gain small — filter ignores measurements
- Output very smooth but slow to respond
- Risk: filter believes itself even when wrong (divergence)

Project EKF #1 (odom) diagonal — exact values:

Index	Component	R_{ii}	Interpretation
0	x	0.05	22 cm / $\sqrt{s}$ position uncertainty growth
1	y	0.05	same
2	z	0.06	higher (vertical bouncing on terrain) — ignored in 2D mode
3	ϕ (roll)	0.03	0.17 rad / $\sqrt{s}$
4	θ (pitch)	0.03	same
5	ψ (yaw)	0.06	0.24 rad / $\sqrt{s}$ — larger because wheel-baseline error
6	$\dot{x}$	0.025	0.16 (m/s) / $\sqrt{s}$ — forward vel uncertainty
7	$\dot{y}$	0.025	holonomic constraint enforces, but kept symmetric
8	$\dot{z}$	0.04	vertical (ignored in 2D)
9	$\dot{\phi}$	0.01	angular vel small uncertainty
10	$\dot{\theta}$	0.01	same
11	$\dot{\psi}$	0.02	yaw rate uncertainty
12	$\ddot{x}$	0.01	accel uncertainty
13	$\ddot{y}$	0.01	same
14	$\ddot{z}$	0.015	gravity residual

Project EKF #2 (map) diagonal — post-debug final values:

Index	Component	R_{ii}	vs EKF#1	Reason
0	x	0.05	same	RTK GPS at 1cm σ pulls fast; moderate Q stops state lock-in
1	y	0.05	same	same
5	ψ (yaw)	0.001	60× tighter	lock yaw, kill $P[\psi, x]$ coupling that drags yaw on GPS x,y updates
6	$\dot{x}$	0.001	25× tighter	non-holonomic: $v_x \approx 0$ unless commanded
7	$\dot{y}$	0.001	25× tighter	non-holonomic: $v_y \equiv 0$ for diff-drive
11	$\dot{\psi}$	0.001	20× tighter	lock ω_z , anchored by wheel + LiDAR $\Delta\theta$

Why EKF #2 needs tight velocity / yaw R (post-debug rationale):

- Old reasoning (loose R) assumed consumer GPS at $\sigma \approx 0.5$ m. With RTK at $\sigma = 1$ cm, filter no longer needs to “flex” to absorb large GPS jumps.
- Loose $Q[\psi]$ lets GPS x,y updates pull yaw via cross-covariance $P[\psi, x]$. Measured: 22° stationary yaw walk in 50 s with $Q[\psi] = 0.06$.
- Loose $Q[\dot{y}]$ allows v_y state to drift; on diff-drive that is unphysical. Measured: v_y ran to +7.33 m/s before being clamped by GPS feedback. Caused 18 m Y drift.
- GPS rejection threshold raised to 100σ to ensure RTK is always trusted (no realistic outlier scenario at 1 cm noise).

Wheel v_y row enabled. `odom0_config[7]: false → true.`
diff_drive_controller publishes $v_y = 0$ with cov 0.001. Without fusing this,

EKF #2 had no measurement source for v_y ; only IMU accel (now disabled) or process noise constrained it. Result was velocity runaway.

IMU linear acceleration disabled in EKF #2. `imu0_config[12..14]: false`. Even with `bias_mean=0`, residual accel noise integrates to velocity drift. Wheel + GPS provide all needed constraints in map frame; accel adds no information.

Tuning rule of thumb:

1. Start with project values
2. Drive a known trajectory
3. Plot innovation: $\nu_t = z_t - h(\bar{\mu}_t)$
4. If innovation grows over time (filter drifts) $\rightarrow$ increase R
5. If innovation noisy/jumps (filter overshooting) $\rightarrow$ decrease R or increase Q
6. Validate with NIS test (see Validation section)

Off-diagonal entries: Project keeps R_t diagonal (zero off-diagonals). This assumes process noise is uncorrelated across state dimensions. A more accurate model could have $R_{xx} \neq 0$ since position and velocity errors are coupled. `robot_localization` accepts this but most implementations leave it diagonal.

Why Yaw $R_{\psi\psi}$ is 0.06 (Twice $R_{\phi\phi}$)

The wheel-encoder yaw estimate uses $\omega = (v_R - v_L)/b$ where b is the baseline. A 1% baseline error becomes a 1% systematic yaw rate error — accumulating linearly over distance. Roll and pitch from IMU gravity vector have no such systematic source — only zero-mean noise. Hence the higher process noise on yaw acknowledges that even with a perfect IMU, yaw can drift due to wheel calibration error.

9 Initial Estimate Covariance Σ_0

`initial_estimate_covariance`

Type: 15×15 matrix **Project values:** see below

What: The covariance Σ_0 at filter startup, before any measurement.

Math role: Just the initial value of Σ_t for $t = 0$. Affects the early Kalman gain:

$$K_0 = \Sigma_0 H_0^\top (H_0 \Sigma_0 H_0^\top + Q_0)^{-1}$$

Large $\Sigma_0 \rightarrow$ filter trusts first measurement strongly. Small $\Sigma_0 \rightarrow$ filter resists initial measurements.

Intuition: “How much do I know about my starting state?” If you start at known origin, $\Sigma_0 \approx 0$ (use `1e-9`). If you start at unknown location, Σ_0 large.

Project EKF #1 (odom): all diagonal entries `1e-9`.

Why: The robot is assumed to start at $(0, 0, 0)$ in odom frame. Origin is the robot’s startup pose by definition.

Project EKF #2 (map): diagonal `[1.0, 1.0, 1e-9, 1.0, 1.0, 1e-9, 1.0, 1.0, ..., 1.0]`

Why: Map-frame position (x, y) initially *unknown* until first GPS fix. $\Sigma_{xx}^{(0)} = 1.0 \text{ m}^2$ allows the first GPS measurement (covariance 0.26 m^2) to dominate via Kalman gain:

$$K \approx \frac{1.0}{1.0 + 0.26} \approx 0.79$$

Filter snaps to GPS quickly. After a few fixes, Σ shrinks below GPS variance; gain drops.

Effect of decreasing initial covariance below sensor noise: filter “trusts” an unknown initial pose more than the actual measurement — output anchored to wrong initial value, drifts away.

Effect of increasing way past needed: no harm; filter converges to correct value within a few cycles regardless.

Tuning rule: Set diagonal entries to $1\text{e-}9$ for components you start at zero (or known value), and to a few σ^2 of the sensor that bounds them otherwise.

10 Control Input Parameters (Local EKF Only)

EKF #1 and #3 use `cmd_vel` as a control input to improve prediction between sensor updates.

use_control

Type: bool **Project value:** `true` (EKF #1, #3) / `false` (EKF #2)

What: Whether the EKF subscribes to `cmd_vel` and uses it as control input u_t in the motion model.

Math role: With control, the prediction is:

$$\bar{\mu}_t = g(u_t, \mu_{t-1})$$

where g depends on commanded (v, ω) . Without control, u_t is treated as zero or constant velocity — prediction degenerates to constant-velocity model.

Intuition: “I know what I told the robot to do; use that to predict.” Helps when sensors are silent or stale.

Effect:

- `true`: prediction follows command, filter snappier
- `false`: relies entirely on past state + sensors. Smoother in steady-state but worse during transients.

Project rationale: Local EKF benefits from `cmd_vel` because wheel feedback has latency. Global EKF doesn’t because it operates on already-fused local odometry.

control_config

Type: list of 6 booleans, $[\dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi}]$ **Project value:** `[true, false, false, false, false, true]`

What: Which fields of `cmd_vel` are valid commands.

Project value: only $\dot{x}$ (forward velocity) and $\dot{\psi}$ (yaw rate). Diff-drive constraint — robot cannot command lateral velocity.

acceleration_limits / deceleration_limits

Type: list of 6 doubles, m/s^2 or rad/s^2 **Project value:** `[1.0, 0, 0, 0, 0, 1.5]`

What: Maximum (de)acceleration the robot can physically achieve.

Math role: Used to limit how fast the predicted velocity can change toward the commanded velocity:

$$\bar{v}_t = \text{clamp}(\bar{v}_t, v_{t-1} - a_{\max}\Delta t, v_{t-1} + a_{\max}\Delta t)$$

Intuition: “This robot has mass. It can’t change velocity instantly.”

Project values: 1.0 m/s² linear, 1.5 rad/s² angular — modest values matching small ground robot.

Failure mode: Set too low → filter doesn't predict sharp commanded changes, lags. Set too high → filter believes robot can teleport, doesn't smooth.

acceleration_gains / deceleration_gains

Type: list of 6 doubles in [0,1] **Project value:** [0.8, 0, 0, 0, 0, 0.9]

What: How fast the robot is expected to ramp acceleration toward commanded value. 1.0 = instant, 0.0 = no response.

Math role: Models actuator dynamics. Predicted acceleration:

$$a_t = g_a \cdot a_{\text{commanded}}$$

Intuition: “Even commanded acceleration is not delivered fully — there's a control loop in the motors.”

Project values: 0.8 linear, 0.9 yaw — 80–90% of commanded acceleration is realised.

control_timeout

Type: double, seconds **Project value:** 0.2

What: If no `cmd_vel` arrives within this time, control input is treated as zero.

Effect: Prevents stale commands from biasing prediction when teleop disconnects.

11 Why GPS is Excluded from EKF #2 (Covariance Cross-Term Leak)

This section explains the engineering reason behind this project's most counter-intuitive design decision: removing GPS from EKF #2. Read carefully before re-enabling.

11.1 The Cross-Covariance Problem

The EKF state covariance Σ is a dense 15×15 matrix. Off-diagonal entries $\Sigma_{x,\psi}$ and $\Sigma_{y,\psi}$ encode statistical correlation between position and yaw — “if I learn something about x , what does it tell me about ψ ?”

These cross-terms become non-zero whenever the robot moves with $\omega \neq 0$. The motion-model Jacobian:

$$G_t = \begin{bmatrix} 1 & 0 & -v \sin(\mu_\theta) \Delta t \\ 0 & 1 & v \cos(\mu_\theta) \Delta t \\ 0 & 0 & 1 \end{bmatrix}$$

has non-zero $\partial g_x / \partial \psi$ and $\partial g_y / \partial \psi$. Predicted covariance $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + R_t$ inherits these cross-terms. After any curved motion, $\Sigma_{x,\psi}, \Sigma_{y,\psi} \neq 0$.

11.2 The Leak Path

GPS measures only $[x, y]$. Observation Jacobian:

$$H_{\text{GPS}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & \dots \\ 0 & 1 & 0 & 0 & 0 & 0 & \dots \end{bmatrix}$$

Kalman gain $K_t = \bar{\Sigma}_t H_t^\top S_t^{-1}$ has a row for *every* state component, including yaw. The yaw-row entries:

$$K_{\psi,x} = \frac{\Sigma_{\psi,x}}{\Sigma_{xx} + Q_{\text{GPS}}}, \quad K_{\psi,y} = \frac{\Sigma_{\psi,y}}{\Sigma_{yy} + Q_{\text{GPS}}}$$

Yaw correction:

$$\Delta\psi = K_{\psi,x} \nu_x + K_{\psi,y} \nu_y$$

where $\nu = z_{\text{GPS}} - h(\bar{\mu}_t)$ is the innovation.

GPS noise of 0.5 m produces non-zero innovation every cycle. Yaw moves — even though GPS never observed yaw.

11.3 Numerical Example (Project Values)

Realistic state after some driving:

$$\Sigma_{xx} = 0.30 \text{ m}^2, \quad \Sigma_{\psi\psi} = 0.02 \text{ rad}^2, \quad \Sigma_{x,\psi} = 0.04$$

GPS noise spike $\nu_x = 0.5 \text{ m}$, $Q_{\text{GPS}} = 0.26 \text{ m}^2$ (your measured GPS variance):

$$K_{\psi,x} = \frac{0.04}{0.30 + 0.26} \approx 0.071$$

$$\Delta\psi = 0.071 \cdot 0.5 \approx 0.036 \text{ rad} \approx 2$$

A single GPS fix moved yaw by 2. At 10 Hz GPS, sustained walk emerges. Measured drift in this project: 3.56/s.

11.4 Compounding via VO/LiDAR

Joseph form covariance update:

$$\Sigma_{\psi,x}^{\text{new}} = \Sigma_{\psi,x} - K_{\psi,x} \Sigma_{xx}$$

reduces but doesn't zero the cross-term. Next predict step regrows it through G_t .

Worse: VO and LiDAR observe *both* position and yaw. Their H matrix has rows for x, y, ψ . The Kalman gain spreads VO/LiDAR's x -correction into yaw via the now-shaped covariance. Each multi-sensor cycle amplifies the leak.

11.5 Why the Canonical Tutorial Doesn't Suffer

Tom Moore's reference example assumes an absolute yaw observation:

- Dual-antenna RTK GPS providing measured heading
- Reliable magnetometer with hard/soft-iron calibration
- External heading reference (compass, sun sensor)

That observation has small Q_ψ . Kalman gain heavily weights the direct yaw measurement, dominating the indirect leak from cross-terms. Yaw stays bounded.

This project has none of those. Gazebo IMU has no magnetometer; “absolute” yaw is integrated gyro — drifts > 150 in 2 minutes by itself. Nothing fights the leak.

11.6 The Sensor Consensus Solution

Remove GPS from EKF #2. Both EKFs now process identical sensor sets:

$$\mu_t^{(\text{global})} \approx \mu_t^{(\text{local})} \implies T_{\text{map}}^{\text{odom}} \approx I$$

Map frame stays near-identity to odom frame. No GPS-driven cross-term injection. Map frame angularly stable.

Empirical evidence in this project:

- Earlier config (GPS in EKF #2): map yaw drifted at 3.56/s, accumulating > 200 in 2 minutes — catastrophic for navigation.
- Current config (GPS removed): map→odom transform stays within ± 0.1 m and ± 1 over 2-minute run.
- Disable GPS → drift disappears. Re-enable → drift returns.

11.7 Trade-Off

Aspect	GPS in EKF #2	GPS excluded (project)
Yaw stability	drifts $\sim 3.56/\text{s}$	rock solid
Global anchoring	yes (cm-m accuracy)	no (drifts with odom)
Re-anchorable to lat/lon	yes	no, only spawn-relative
Best for	open outdoor + RTK + magnetometer	indoor / GPS-denied / short-range

For most navigation tasks, a stable heading matters more than centimetric global position. A robot drifting 3.56/s of yaw will spin in circles trying to reach waypoints. A robot drifting 0.5 m/min of global position can still navigate a 50 m mission.

11.8 If You Need Global Anchoring Back

Four options:

1. **Add a strong yaw observation:** dual-antenna RTK, calibrated magnetometer (with hard/soft-iron calibration), or sun sensor. Strong Q_ψ measurement drowns out the leak.
2. **Diagonalise Σ post-GPS-update:** zero out $\Sigma_{x,\psi}$ and $\Sigma_{y,\psi}$ after each GPS correction. Hacky but effective. Requires custom EKF code (not available in stock `robot_localization`).
3. **Loosely-coupled GPS:** run a separate filter that consumes only EKF #2 output and GPS, publish a `utm→map` transform without polluting EKF #2's state.
4. **Periodic re-anchoring:** when GPS confidence is high (RTK fix), trigger a one-shot pose reset of EKF #2 instead of continuous fusion. Resets cross-term to zero, breaks the compounding cycle.

11.9 Where to Look in YAML

Compare the actual project files:

- `src/husarion_ugv_description/config/dual_ekf_navsat.yaml` — canonical-style file with three EKFs, originally had GPS in EKF #2
- `src/husarion_ugv_description/config/baseline_ekf.yaml` — current production config, GPS not in EKF #2 input list

- `src/husarion_ugv_description/config/hybrid_qf_ekf.yaml` — experimental quantum filter variant

The relevant block in EKF #2's section (post-debug, final working):

```

1 ekf_filter_node_map:
2   ros__parameters:
3     world_frame: map
4     use_control: false
5
6     # Wheel: vx, vy (kinematic constraint), vyaw
7     odom0: /diff_drive_controller/odom
8     odom0_config: [false, false, false,
9                   false, false, false,
10                  true, true, false,    # vx, vy, vz
11                  false, false, true,   # vroll, vpitch, vyaw
12                  false, false, false]
13     odom0_differential: false
14
15     # GPS: x, y absolute (RTK, ~1cm noise)
16     odom1: /odometry/gps
17     odom1_config: [true, true, false,
18                   false, false, false,
19                   false, false, false,
20                   false, false, false,
21                   false, false, false]
22     odom1_pose_rejection_threshold: 100.0 # RTK can't outlier
23     odom1_differential: false
24
25     # LiDAR: yaw absolute, differential mode (      between samples)
26     odom2: /odometry/lidar
27     odom2_config: [false, false, false,
28                   false, false, true,    # yaw only
29                   false, false, false,
30                   false, false, false,
31                   false, false, false]
32     odom2_differential: true
33
34     # IMU: angular velocity only (no accel - bias integrates to drift)
35     imu0: /imu/data/covariance_fixed
36     imu0_config: [false, false, false,
37                  false, false, false,
38                  false, false, false,
39                  true, true, true,    # vroll, vpitch, vyaw
40                  false, false, false] # accel DISABLED
41     imu0_differential: false
42
43     # Tight process noise: lock yaw + velocities (non-holonomic)
44     # Q[x,y]      = 0.05
45     # Q[yaw]      = 0.001
46     # Q[vx,vy]    = 0.001
47     # Q[vyaw]     = 0.001

```

Launch file remap (the topology fix):

```

1 # baseline_ekf.launch.py, navsat_transform node:
2 remappings=[
3   ("imu", "imu/data/covariance_fixed"),
4   ("gps/fix", "gps/fix/covariance_fixed"),
5   ("odometry/filtered", "odometry/global"), # NOT odometry/local
6 ],

```

12 NavSat Transform Parameters

frequency (navsat)

Type: double, Hz **Project value:** 10.0

What: How often `navsat_transform_node` publishes `/odometry/gps`.

Effect: Should match GPS update rate. Higher wastes CPU; lower drops fixes.

delay

Type: double, seconds **Project value:** 3.0

What: Wait this long after first GPS fix before computing the datum transform.

Math role: Allows the EKF to stabilise its initial estimate before locking the GPS-to-map alignment. The datum is computed from:

$$T_{\text{map}}^{\text{UTM}} = \text{Yaw}(\psi_0) \cdot \text{Translate}(-x_0^{\text{UTM}}, -y_0^{\text{UTM}})$$

where ψ_0 is taken from the IMU after `delay` seconds.

Intuition: “Wait for everything to settle before snapshotting the world.”

Effect of small delay: initial heading from IMU may be noisy — map frame misaligned.

Effect of large delay: robot may have moved before datum set — origin offset.

magnetic_declination_radians

Type: double **Project value:** 0.0

What: Angle between true north and magnetic north at robot’s location.

Math role: Used to align the IMU’s magnetometer-derived yaw with true north (UTM grid):

$$\psi_{\text{true}} = \psi_{\text{magnetic}} + \delta_{\text{declination}}$$

Intuition: “Compass doesn’t point to true north.” Varies geographically: -20° to $+20^\circ$ globally.

Project value 0.0: acceptable in simulation (Gazebo provides true north). For real robot at Edinburgh: declination $\approx -1.2^\circ \approx -0.021$ rad. Look up at <https://www.ngdc.noaa.gov/geomag-web/>.

Failure mode: Wrong declination $\rightarrow$ map frame rotated $\rightarrow$ all GPS-derived positions twisted by that angle. Robot “walks sideways” relative to map.

yaw_offset

Type: double, radians **Project value:** 0.0

What: Rotation between IMU’s zero-yaw direction and the conventional $+x$ axis (East in ENU).

Math role: If IMU’s $\psi = 0$ points North, but UTM East is the $+x$ axis, you need `yaw_offset` $= -\pi/2$.

Project value 0.0: simulator IMU is configured so $\psi = 0$ aligns with $+x$ East.

Failure mode: Most common config error. Robot drives east according to filter but actually drives north. Diagnose by driving in commanded direction and checking GPS.

zero_altitude

Type: bool **Project value:** true

What: Force altitude in published odometry to zero, regardless of GPS altitude.

Effect: For a planar robot, this prevents GPS altitude noise (1.23 m² in this project!) from corrupting the z estimate.

Project rationale: Lynx is a ground robot. Altitude irrelevant.

publish_filtered_gps

Type: bool **Project value:** true

What: Publishes the EKF's filtered estimate back as a `NavSatFix` (lat/lon).

Effect: Useful for visualisation in mapping tools (Google Maps, Foxglove). No effect on filter behavior.

use_odometry_yaw

Type: bool **Project value:** false

What: If true, takes initial heading from `/odometry/filtered` (i.e., wheel + IMU fused yaw) instead of raw IMU.

Effect:

- **false** (project): use IMU yaw at startup — absolute reference (assuming magnetometer)
- **true**: use fused yaw, which is initially zero (odom frame origin)

Project chose **false**: relies on IMU's absolute heading sensor for global alignment.

wait_for_datum

Type: bool **Project value:** false

What: If true, waits for an explicit `datum` parameter before computing transforms. If false, uses first GPS fix as datum automatically.

Effect:

- **false** (project): convenient for tests — start anywhere, first fix becomes origin
- **true**: requires `datum`: [lat, lon, yaw] explicitly. Repeatable across runs — map frame consistent across launches.

Production setup: should use **true** with a known datum so map frame is consistent across robot restarts.

broadcast_cartesian_transform

Type: bool **Project value:** true

What: Whether to publish `utm→world_frame` TF.

Effect: Useful for tools that need UTM coordinates directly. Doesn't affect EKF.

13 Per-Sensor Covariance Override

`odomN_pose_covariance_diagonal / odomN_twist_covariance_diagonal`

Type: list of 6 doubles **Project values:** see below

What: Override the message’s reported covariance with these values.

Math role: Replaces the diagonal entries of Q_t used in the update step:

$$Q_t^{(s)} = \text{diag}(\text{covariance_diagonal})$$

Intuition: “Sensor’s own covariance is wrong — use my measured value instead.”

Project values (EKF #3 fusion):

```

1 # Wheel odom: tight pose/twist (very confident on velocities)
2 odom0_pose_covariance_diagonal: [0.001, 0.001, 0.001, 0.001, 0.001,
   0.001]
3 odom0_twist_covariance_diagonal: [0.001, 0.001, 0.001, 0.001, 0.001,
   0.001]
4
5 # Visual odom: looser pose, looser velocity
6 odom1_pose_covariance_diagonal: [0.01, 0.01, 0.01, 0.01, 0.01, 0.05]
7 odom1_twist_covariance_diagonal: [0.1, 0.1, 0.1, 0.1, 0.1, 0.1]
8
9 # LiDAR odom: tighter than VO
10 odom2_pose_covariance_diagonal: [0.005, 0.005, 0.005, 0.01, 0.01, 0.02]
11 odom2_twist_covariance_diagonal: [0.1, 0.1, 0.1, 0.1, 0.1, 0.1]
```

Why these values:

- **Wheel** 0.001: trusted velocity source, very tight. Note: trusting position to 0.001 is meaningless because we disable position bits.
- **Visual** 0.01–0.05: VO drifts with distance, monocular has scale issues. Larger yaw covariance (0.05) reflects rotation ambiguity.
- **LiDAR** 0.005–0.02: more accurate than VO in structured environments. Tightest non-wheel source.

Tuning order: Use override only when you know the message’s covariance is wrong (very common — many drivers publish placeholder values like all-1s or all-0s).

14 IMU-Specific Parameters

`imuN_remove_gravitational_acceleration`

Type: bool **Project value:** true

What: Subtracts gravity from accelerometer readings before fusing.

Math role: Raw accel reads:

$$\tilde{a}^b = a_{\text{true}}^b - R_w^b g$$

With this flag **true**, the filter computes:

$$a_{\text{linear}}^b = \tilde{a}^b + R_w^b g$$

using the current orientation estimate to rotate gravity into body frame.

Intuition: “Don’t let gravity look like horizontal acceleration when the robot tilts.”

Effect of true:

- Filter receives only motion-induced acceleration
- Robust to terrain pitch/roll

Effect of false:

- Filter receives raw $\tilde{a}^b$ including $-R_w^b g$
- In 2D mode, planar accel includes gravity components when robot pitches
- Sensor model h would need to predict gravity itself

Failure mode: `false` on a sloped robot $\rightarrow$ filter believes constant horizontal acceleration $\rightarrow$ velocity drifts.

15 The Big Picture: How Parameters Compose

15.1 Causal Chain — One Update Cycle

Imagine GPS has just arrived. Trace through what happens:

1. **Predict step uses:**

- `frequency` sets Δt
- `two_d_mode` clamps non-planar dims
- `use_control`, `control_config`, `acceleration_limits` shape $g(u_t, \mu_{t-1})$
- `process_noise_covariance` provides R_t
- Result: $\bar{\mu}_t, \bar{\Sigma}_t$

2. **Sensor message arrives on /odometry/gps:**

- `odom1` topic identifies it
- `odom1_config` extracts which dims to use $\rightarrow$ builds H_t
- Message's pose covariance becomes Q_t (or override via `odom1_pose_covariance_diagonal`)
- `odom1_differential`, `_relative` transform the value if set

3. **Gating:**

- Compute innovation $\nu_t = z_t - h(\bar{\mu}_t)$
- Mahalanobis $d_M^2 = \nu_t^\top S_t^{-1} \nu_t$
- Compare against `odom1_pose_rejection_threshold`
- Reject if too far

4. **Update step:**

- Kalman gain $K_t = \bar{\Sigma}_t H_t^\top S_t^{-1}$
- State update $\mu_t = \bar{\mu}_t + K_t \nu_t$
- Covariance update $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$

5. **Publish:**

- TF if `publish_tf`
- `/odometry/filtered_map` (or `/odometry/filtered`)
- Diagnostics if `print_diagnostics`

15.2 The Trust Triangle

Three covariances determine the filter’s behavior:

- Σ_0 (initial trust in self)
- R_t (ongoing trust in motion model)
- Q_t (trust in each sensor)

The Kalman gain magnitude:

$$K_t \sim \frac{\bar{\Sigma}_t}{\bar{\Sigma}_t + Q_t}$$

controls measurement weight. The relative scale of R_t vs Q_t determines steady-state behavior:

Setting	Effect
R_t large, Q_t small	Trust sensors. Output noisy. Fast response.
R_t small, Q_t large	Trust model. Output smooth. Slow response.
$R_t \sim Q_t$	Balanced. Typical for well-tuned filter.
$R_t \gg Q_t$ and model wrong	Filter chases noisy sensors, output unstable.
$R_t \ll Q_t$ and model wrong	Filter ignores sensors, drifts away from truth.

15.3 Why EKF #1 and EKF #2 Have Different R_t

The Key Insight

Both EKFs run on the same robot. They differ only in which sensors they fuse and what frame they operate in. Why are their process noise covariances different by $10\times$?

Answer: EKF #1 (odom) sees only continuous, high-rate sensors (wheel + IMU). Between sensor ticks, very little time passes ($\Delta t = 0.02$ s), so process noise per step is small.

EKF #2 (map) operates between GPS fixes (10 Hz $\rightarrow$ 0.1 s). During those 100 ms, the robot can drift significantly relative to the global frame. To prepare for the GPS update, $\bar{\Sigma}_t$ must grow large enough that K_t is high when GPS arrives. Hence R_t is set higher to inflate $\bar{\Sigma}$ faster.

The math: between fixes,

$$\bar{\Sigma}_t = \bar{\Sigma}_{t-1} + R \Delta t$$

We want $\bar{\Sigma}_{xx}$ to be comparable to $Q_{xx}^{\text{GPS}} = 0.26$ when GPS arrives. With $\Delta t = 0.1$ and $R_{xx} = 0.5$: Σ grows by 0.05 per cycle, reaches 0.5 in 10 cycles — well above GPS variance. Kalman gain is high; filter snaps to GPS.

If R_{xx} were 0.05 like EKF #1, Σ would only grow to 0.005 between fixes — way below GPS variance. Kalman gain $\approx 0.005/0.265 \approx 2\%$. Filter would barely move toward GPS. Result: GPS rejected as outlier, map estimate drifts.

16 Tuning Workflow

16.1 Initial Sanity Check

1. All sensors publishing? (`ros2 topic hz`)
2. All TFs resolving? (`ros2 run tf2_tools view_frames`)
3. Each sensor’s `frame_id` correct in URDF?

4. EKF receiving messages? (/diagnostics)

16.2 Stationary Test

Robot should be still:

- Output velocity ≈ 0
- Output position drifts ≈ 0 over minutes
- Output yaw stable to < 0.01 rad/min

Failures:

- Velocity drifts $\rightarrow$ accelerometer bias not cancelled. Lower $R_{\ddot{x}}$, increase Q for accel.
- Yaw drifts fast $\rightarrow$ gyro bias too large. Use `imu_filter_madgwick` for orientation, increase its bias estimation gain.

16.3 Straight-Line Test

Drive 10 m in straight line. Check:

- Output position matches (within sensor tolerance)
- No spurious yaw drift (filter shouldn't "believe" robot turned)

16.4 Turn-in-Place Test

Spin 360° at 0.5 rad/s. Check:

- Output integrates to ≈ 0 net displacement
- Yaw integrates to $\approx 2\pi$

Mismatch in yaw indicates baseline calibration error or gyro scale factor.

16.5 NIS Validation

Compute Normalised Innovation Squared per measurement:

$$\text{NIS}_t = \nu_t^\top S_t^{-1} \nu_t$$

Should be χ_k^2 distributed where k is measurement dimension.

Average NIS	Diagnosis	Fix
$\approx k$	Well-tuned	—
$\gg k$	Overconfident filter	Increase R_t or Q_t
$\ll k$	Underconfident filter	Decrease R_t or Q_t

16.6 Common Failure Diagnoses

Filter Output Stuck

Symptom: filter ignores GPS, position frozen.

Causes:

1. Initial covariance too small ($1\text{e-}9$ for x, y in EKF #2) $\rightarrow$ Kalman gain near zero. Increase to $1.0+$.
2. Rejection threshold too tight $\rightarrow$ all GPS rejected. Increase to 5.0 .

3. GPS covariance in message too large → Kalman gain near zero. Override with smaller value.

Filter Output Jumpy

Symptom: position bounces around.

Causes:

1. Rejection threshold too loose → outliers accepted. Decrease.
2. R_t too high → filter chases every measurement. Decrease.
3. Sensor noise underestimated in Q_t . Increase.

Yaw Drifts

Symptom: heading slowly rotates over time.

Causes:

1. Wheel baseline miscalibrated. Recalibrate.
2. IMU yaw bit not enabled in `imuN_config`. Enable.
3. No magnetometer + long run → inevitable. Use loop closure or GPS heading.

TF Errors

Symptom: “Lookup would require extrapolation into future” / “into past”.

Causes:

1. Clock skew. Sync system clocks.
2. `transform_time_offset` set wrong. Try 0.05.
3. Sensor timestamps in wrong unit. Check `ros2` topic echo.

17 Quick Reference Tables

17.1 All Parameters at a Glance

Parameter	Type	Project value	Effect of increasing value
<code>frequency</code>	Hz	50.0	smoother, more CPU
<code>sensor_timeout</code>	s	0.1	wait longer for laggy sensors
<code>two_d_mode</code>	bool	true	lock z, roll, pitch
<code>world_frame</code>	string	odom/map	determines TF role
<code>publish_tf</code>	bool	true	node owns TF link
<code>use_control</code>	bool	true	use <code>cmd_vel</code> for prediction
<code>control_timeout</code>	s	0.2	ignore stale commands
<code>acceleration_limits</code>	m/s ²	[1.0, ..., 1.5]	allow sharper accel
<code>acceleration_gains</code>	[0,1]	[0.8, ..., 0.9]	trust commanded accel more
<code>process_noise_covariance</code>	15×15	varies	faster filter, noisier
(R)			
<code>initial_estimate_covariance</code>	15×15	varies	first measurement dominates
(Σ_0)			
<code>odomN_config</code>	15 bool	varies	include more state dims
<code>odomN_differential</code>	bool	false	use deltas, not absolute
<code>odomN_relative</code>	bool	false	first msg = origin
<code>odomN_pose_rejection_threshold</code>	bool	3.0 (GPS)	accept more outliers
<code>imuN_remove_gravitational_acceleration</code>	bool	true	subtract gravity
<code>magnetic_declination_radians</code>	rad	0.0	rotate map frame
<code>yaw_offset</code>	rad	0.0	align IMU 0 with map +x

continued on next page

continued from previous page

Parameter	Type	Project value	Effect of increasing
<code>zero_altitude</code>	bool	true	ignore GPS altitude
<code>wait_for_datum</code>	bool	false	require explicit datum
<code>delay (navsat)</code>	s	3.0	wait longer before locking transform

17.2 Project Configuration Summary — Side-by-Side

Parameter	EKF #1 (odom)	EKF #2 (map, post-debug)
<code>world_frame</code>	odom	map
<code>frequency</code>	50	50
<code>two_d_mode</code>	true	true
<code>use_control</code>	true	false
Wheel odom (config)	v_x, ω_z	v_x, v_y, ω_z
IMU (config)	orient + ω + accel	ω only (no accel)
<code>imu0_differential</code>	false	false
Visual odom	yes (pose, diff)	— (removed)
LiDAR odom	yes (pose+yaw, diff)	yes (yaw only, diff mode)
GPS	—	yes (RTK, $R=10^{-4}$)
GPS rejection threshold	—	100
R_{xx}	0.05	0.05
$R_{\dot{x}\dot{x}}$	0.025	0.001
$R_{\psi\dot{\psi}}$	0.06 (or 0.08)	0.001
$R_{\dot{\psi}\dot{\psi}}$	0.04	0.001
Σ_0^{xx}	1e-9	1e-9
$\Sigma_0^{\psi\dot{\psi}}$	1e-9	1e-9
Map x,y drift (stationary)	—	≤ 3 cm (RTK noise floor)
Map yaw drift (stationary)	—	$\leq 0.1^\circ$ (locked)
Global anchoring	—	GPS (RTK)

Reading the table: EKF #2 uses a leaner sensor set than EKF #1 (no VO, no IMU accel) plus tight velocity/yaw process noise. Three tweaks vs “canonical”: (1) GPS rejection threshold 100σ (RTK noise too low to produce realistic outliers), (2) tight $Q[\psi], Q[\dot{\psi}], Q[\ddot{\psi}]$ (kinematic + cross-cov constraints), (3) navsat node consumes EKF #2 output (*not* EKF #1) to break the circular reference loop documented in `ekf_drift_debug.pdf`.

Closing Notes

Every parameter in `robot_localization` corresponds to a specific term in the EKF equations. The `config` bits build the observation matrix H_t . The `covariance_diagonal` overrides shape the measurement noise Q_t . The `process_noise_covariance` is the motion noise R_t . The `initial_estimate_covariance` is Σ_0 . The `rejection_threshold` parameters implement Mahalanobis gating $d_M^2 < \text{threshold}$. The `world_frame` parameter determines whether the node plays the local or global role in the dual-EKF design.

Tuning is not arbitrary — each knob has a mathematical home. When the filter misbehaves, identify which equation governs the symptom, then adjust the parameter that controls that equation. The relative scale of R to Q determines steady-state Kalman gain; the relative scale of Σ_0 to Q at startup determines initial trust in the first measurement.

Update (post-debug, 2026-05-09): The earlier “deliberate omission of GPS from EKF #2” has been reversed. GPS is now fused in EKF #2 with the following preconditions met:

- URDF GPS noise tightened to RTK grade ($\sigma \approx 1$ cm horizontal). Relay covariance matched at 10^{-4} m^2 .
- LiDAR yaw added in differential mode (`odom2_config[5]=true`, `differential=true`). Provides the absolute yaw observation needed to suppress GPS-driven yaw coupling.
- Wheel $v_y=0$ enabled (`odom0_config[7]=true`). Enforces non-holonomic constraint at the measurement level.
- IMU linear acceleration disabled in EKF #2. Wheel + GPS provide all needed velocity/-position information; accel only injects bias.
- Process noise tightened: $Q[\psi] = Q[\dot{x}] = Q[\dot{y}] = Q[\dot{\psi}] = 0.001$. Non-holonomic constraint + cross-covariance suppression.
- GPS rejection threshold raised to 100σ . RTK is too clean to produce realistic outliers; gating only blocks legitimate corrections.
- `navsat_transform_node` input remapped from EKF #1 (`/odometry/local`, odom frame) to EKF #2 (`/odometry/global`, map frame). Without this remap, GPS in odom frame gets re-transformed by EKF #2's own `map→odom` TF, producing self-consistent garbage that never corrects state. See `report/ekf_drift_debug.pdf` for the 11-step case study.

Result: stationary hold at GPS within 1–3 cm, yaw locked within 0.1° . Original 3.56/s yaw leak observation came from a config with loose $Q[\psi]$ and consumer-grade GPS noise; the same architectural pattern is stable once those underlying issues are fixed.

Combine this document with `EKF_Deep_Dive_Guide.pdf` (theory) for a complete picture: theory explains *why* the equations look the way they do; this document explains *which knob controls what*, grounded in the actual project YAML.